Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

Tris récursifs

1. Diviser pour régner
2. Tri fusion
3. Tri rapide

Diviser pour régner

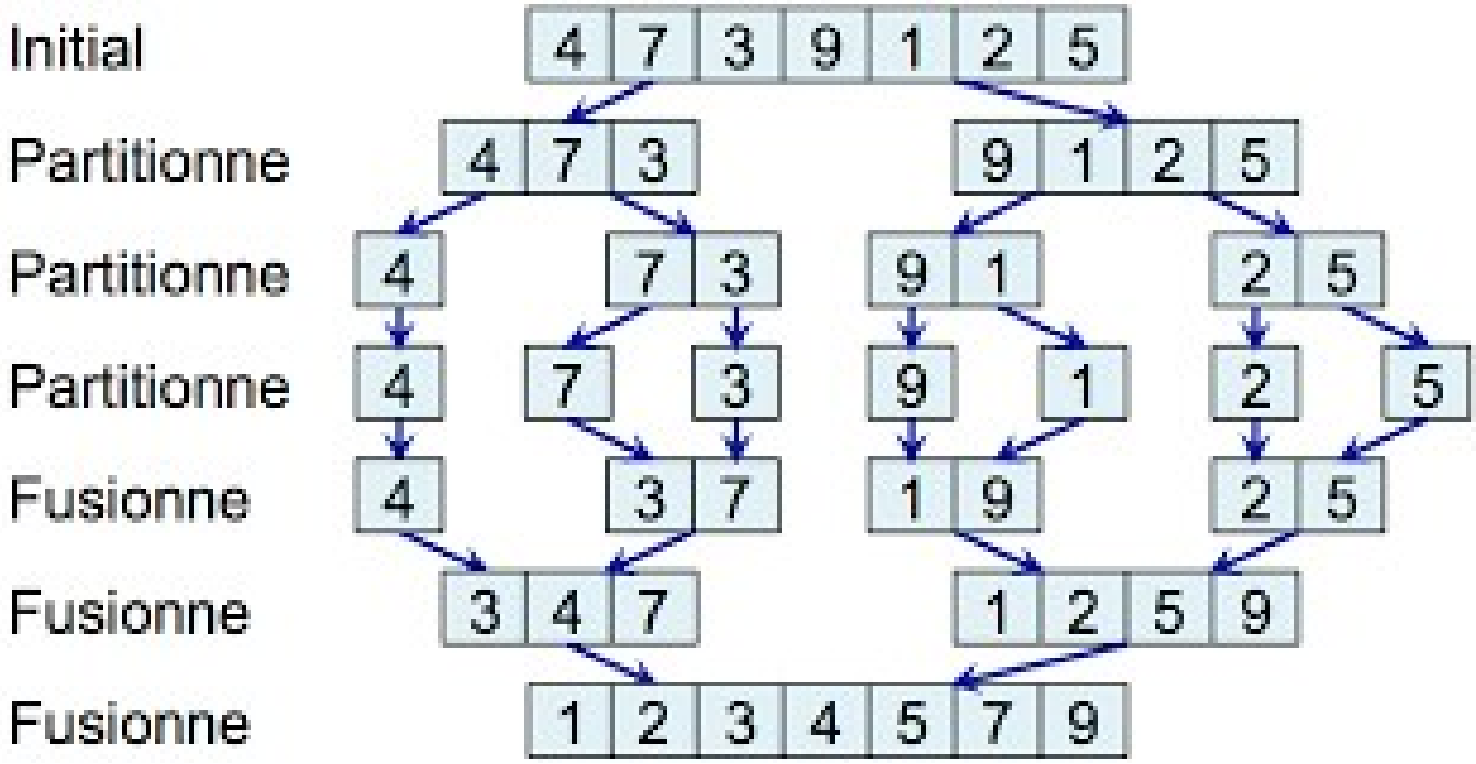
- Trois étapes
  - Diviser le problème de taille n en plusieurs sous-problèmes de tailles plus petites
  - Résoudre les sous-problèmes (généralement de manière récursive)
  - Combiner les solutions des sous-problèmes pour obtenir une solution du problème initial

Tri fusion

- Inventé par John Von Neumann en 1945

- Supposons que l'on dispose d'un algorithme qui construit une liste triée à partir de deux listes triées.
- Une solution appliquant l'approche diviser pour régner est la suivante :
  - Diviser la liste en deux listes de tailles identiques.
  - Trier récursivement les deux sous-listes.
  - Combiner les deux sous-listes triées en une seule liste triée

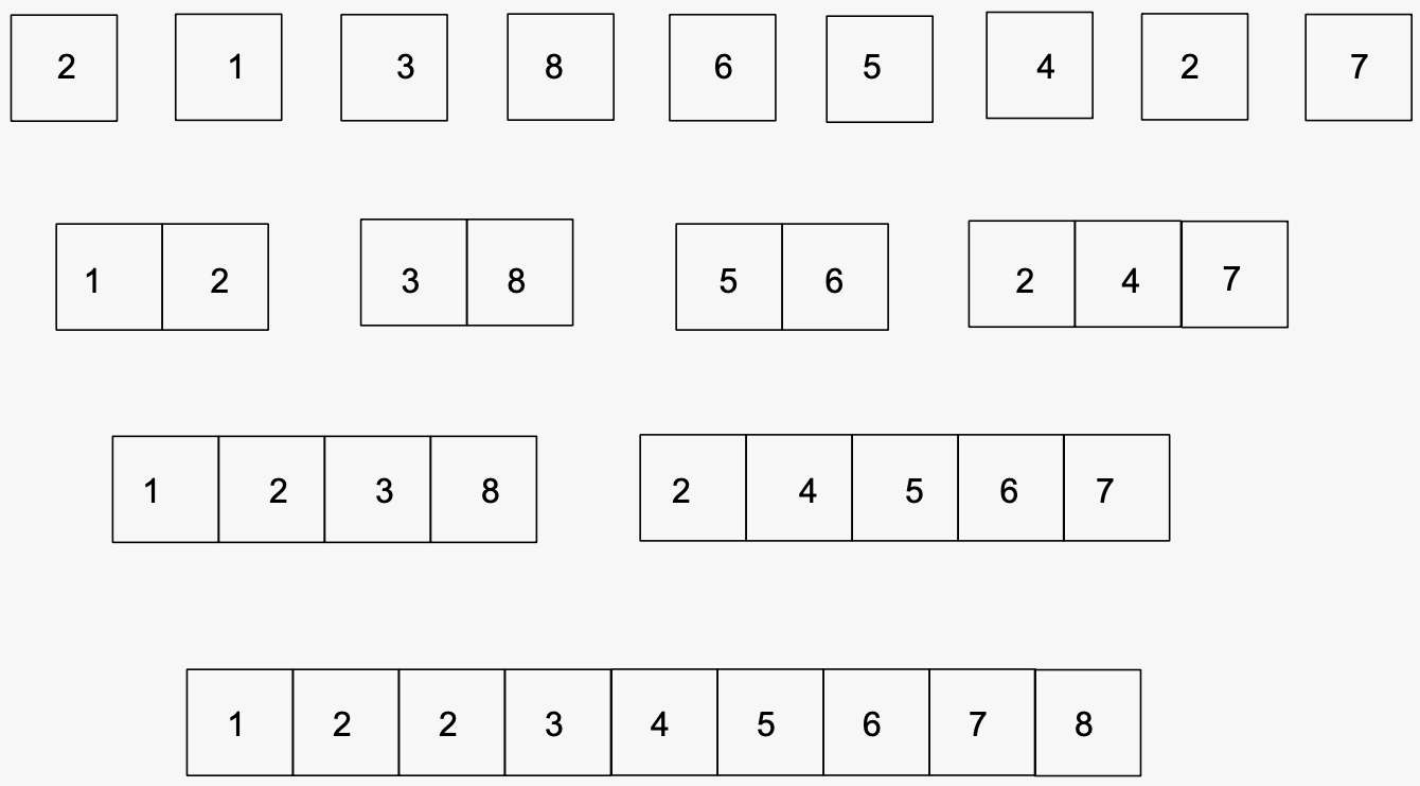
Un exemple du tri fusion récursif



Tri fusion

- Une version plus simple, itérative :
  - Diviser la liste en autant de sous-listes que d'éléments.
  - Combiner deux à deux les sous-listes en les fusionnant

Un exemple du tri fusion itératif



Comment fusionner 2 listes triées ?

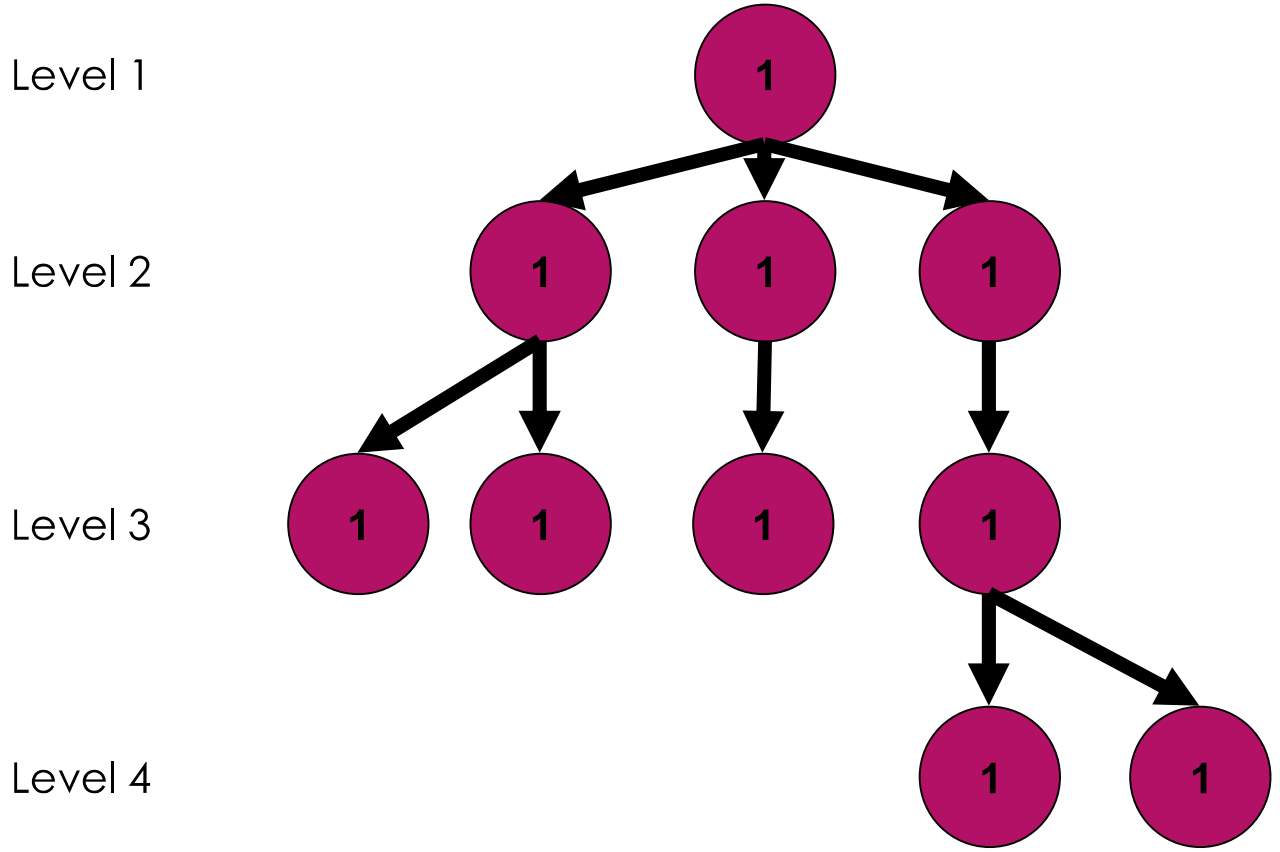
- Les deux listes à fusionner sont triées
- On compare les éléments de chacune des deux listes
- On déplace le plus petit dans la liste fusionnée
- Quand une des deux listes est vide, on déplace les éléments restants de la seconde liste

liste1	liste2	résultat
[3,5,8,9]	[1,2,6,10]	[ ]
[3,5,8,9]	[2,6,10]	[1]
[3,5,8,9]	[6,10]	[1,2]
[5,8,9]	[6,10]	[1,2,3]
[8,9]	[6,10]	[1,2,3,5]
[8,9]	[10]	[1,2,3,5,6]
[9]	[10]	[1,2,3,5,6,8]
[ ]	[10]	[1,2,3,5,6,8,9]
[ ]	[ ]	[1,2,3,5,6,8,9,10]

Arbres binaires de recherche

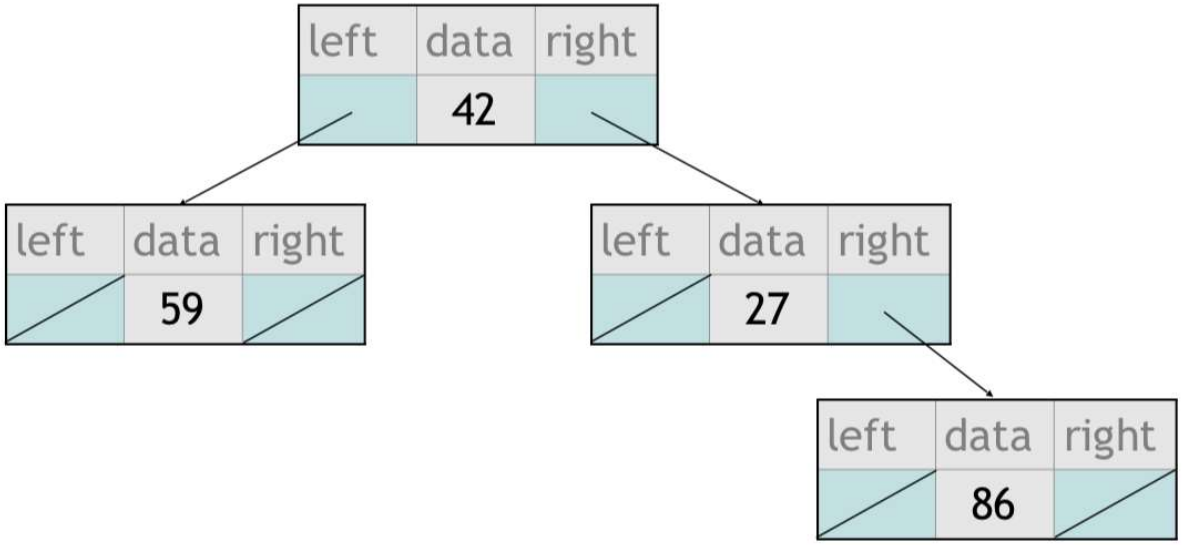
Les arbres

- Graphe acyclique enraciné



Structure d'arbre

- Un nœud est défini par :
  - la valeur qu'il contient,
  - son fils gauche (nœud),
  - son fils droit (nœud également).
- Un arbre contient un pointeur vers le nœud racine de l'arbre.

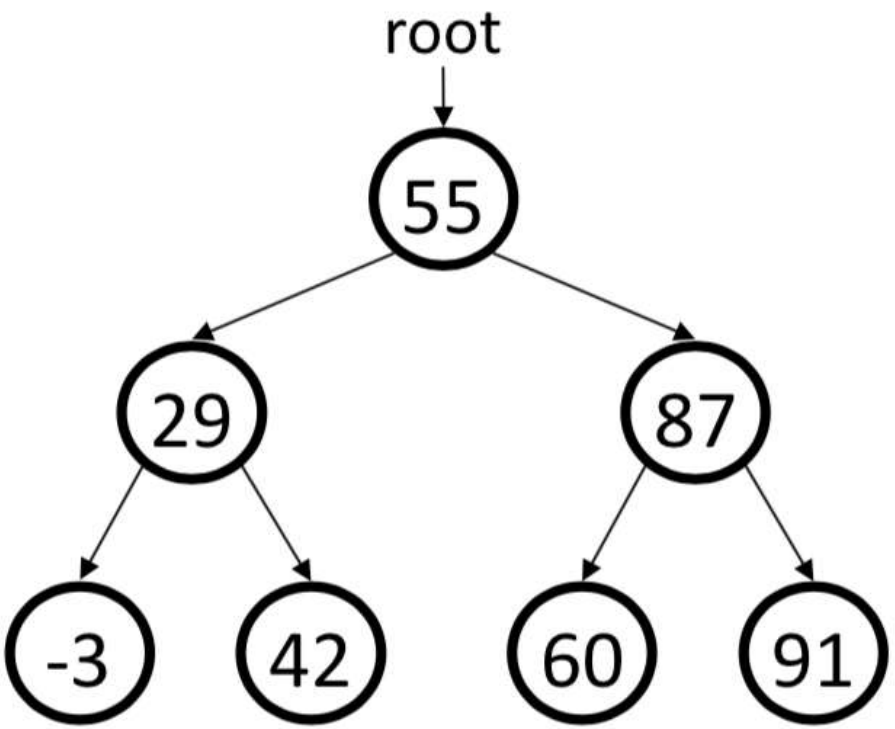


Arbres binaires de recherche

- En informatique, un arbre binaire de recherche (en anglais, binary search tree ou BST) est une structure de données représentant un ensemble ordonné de valeurs (clés).
- Chaque nœud possède une valeur
  - chaque nœud du sous-arbre gauche a une valeur inférieure ou égale,
  - chaque nœud du sous-arbre droit possède une valeur supérieure ou égale
  - les sous-arbres gauche et droit sont également des BST

- Un arbre binaire de recherche permet des opérations rapides pour rechercher une clé, insérer ou supprimer une clé.

Exemple



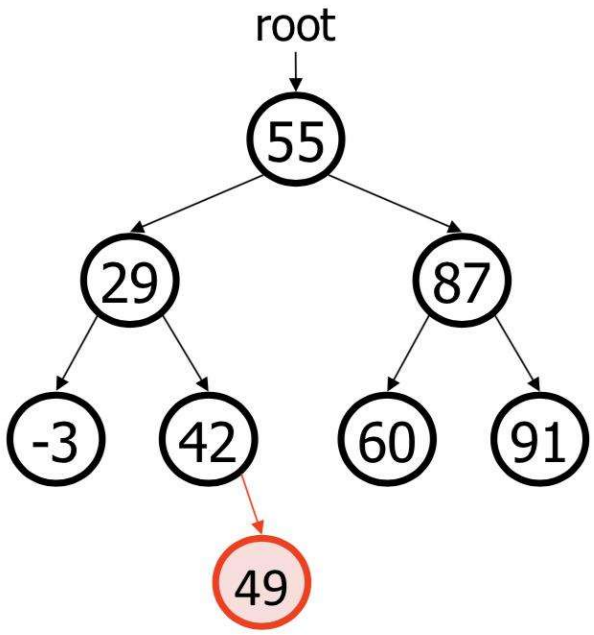
Comment construire un BST ?

- Draw what a binary search tree would look like if the following values were added to an initially empty tree in this order:

50
20
75
98
80
31
150
39
23
11
77

Ajouter un élément

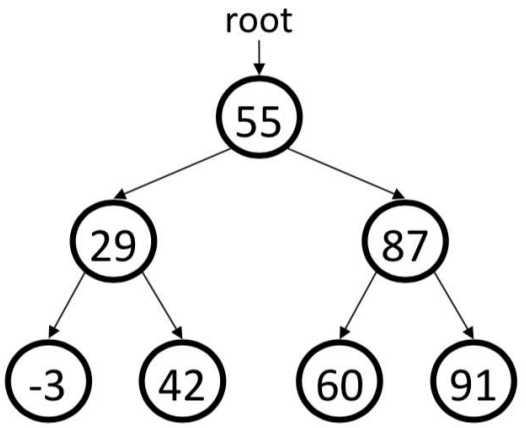
- Ecrire une fonction insertNodeBST(tree_t *bst, int value) qui ajoute le nœud contenant la valeur *value* au bon endroit dans l'arbre BST



Parcours

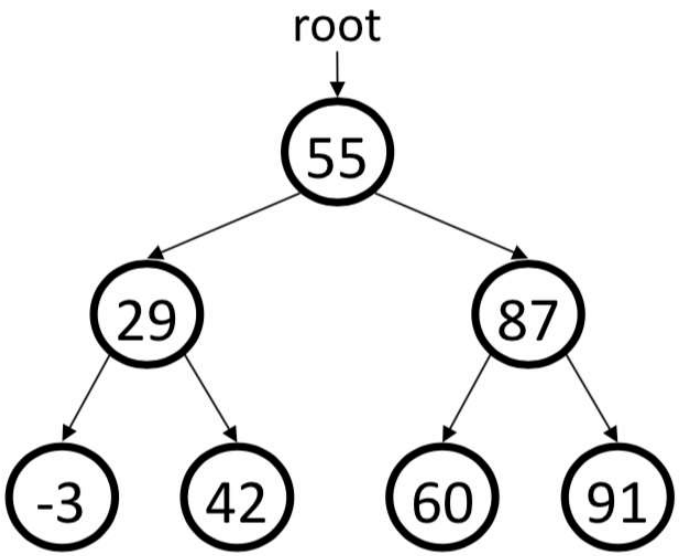
- **traversal**: An examination of the elements of a tree.
  - A pattern used in many tree algorithms and methods

- Common orderings for traversals:
  - **pre-order**: process root node, then its left/right subtrees
  - **in-order**: process left subtree, then root node, then right
  - **post-order**: process left/right subtrees, then root node



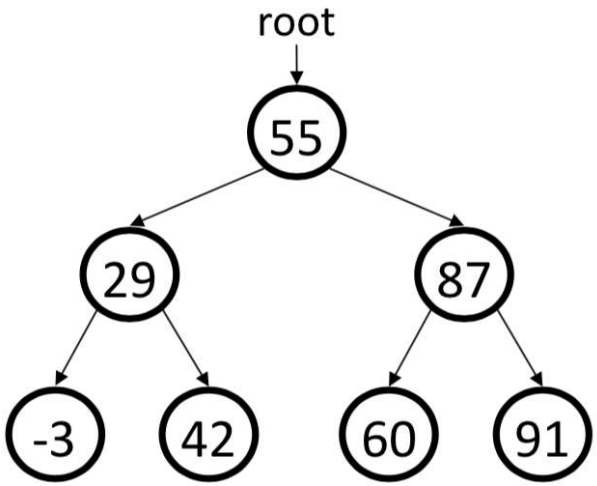
Parcours

- Pre-order : 55 29 –3 42 87 60 91
- In-order : -3 29 42 55 60 87 91
- Post-order : -3 42 29 60 91 87 55



contains

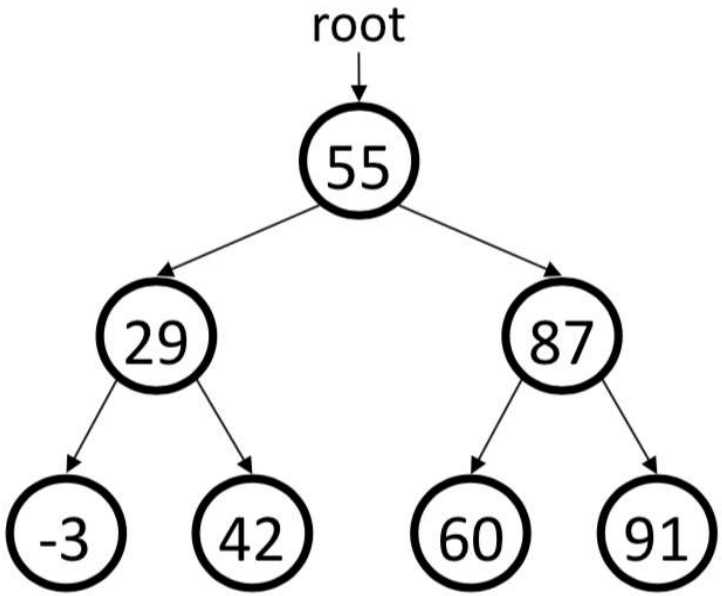
- Write a function **`contains`** that accepts a tree node pointer as its parameter and searches the tree for a given integer, returning `true` if found and `false` if not.
  - `contains(root, 87)` → `true`
  - `contains(root, 60)` → `true`
  - `contains(root, 63)` → `false`



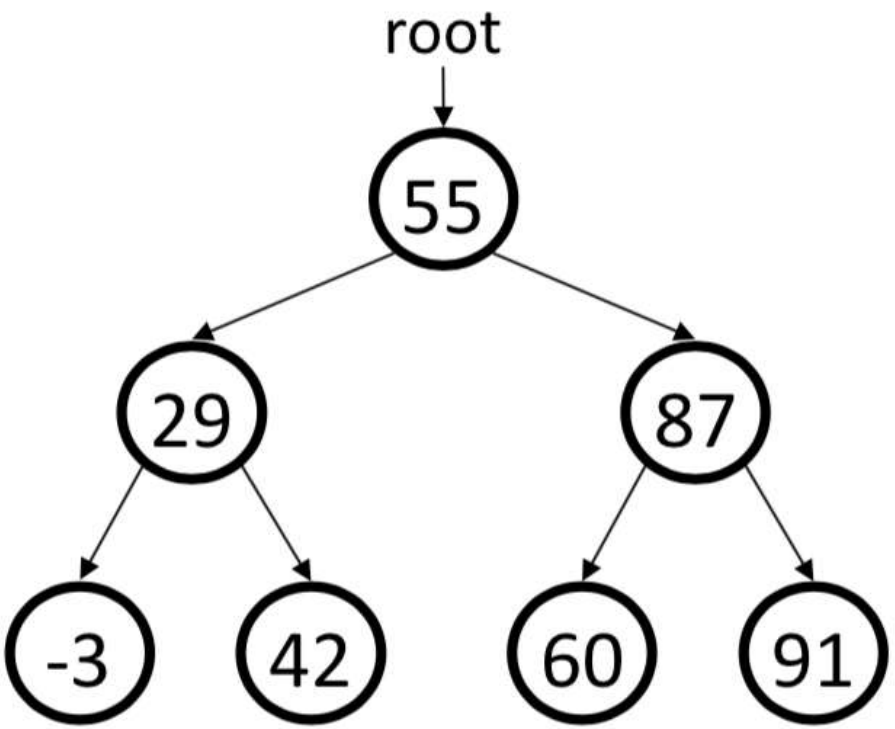
Des fonctions supplémentaires

size

- Write a function named **`size`** that accepts a tree node pointer as its parameter and returns the number of elements of that tree.
  - An empty/null tree is defined as having a size of 0.
  - Example: `size(root)` returns `7`

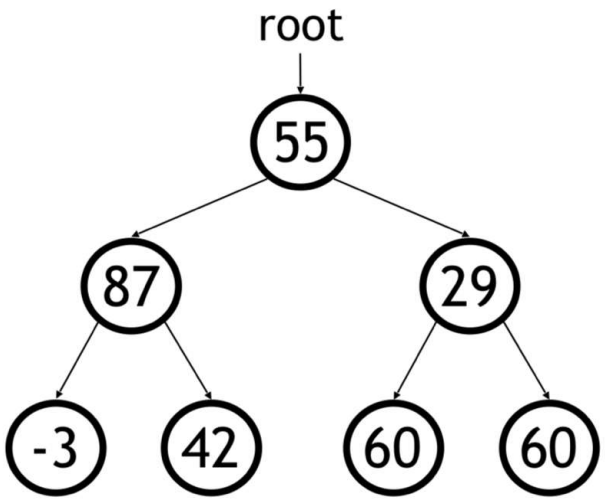


Exemple



height

- Write a function **height** that accepts a tree node pointer as its parameter and returns the height of the tree.
- Height is defined as the longest path from the root to any node, in # nodes.

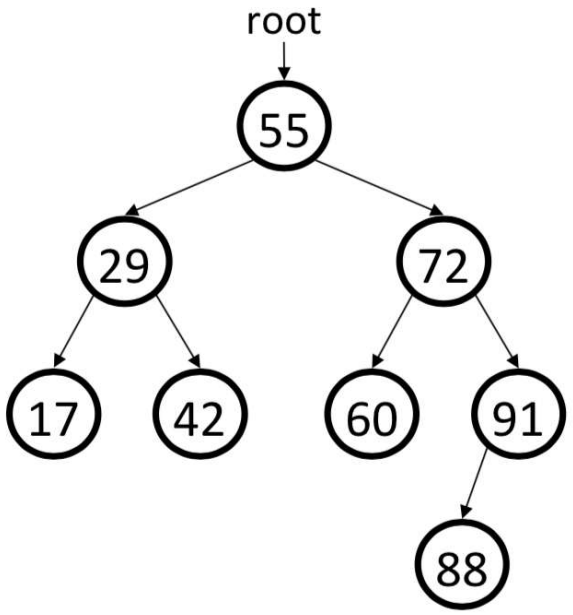


E.g. the height of this tree is **3**.

getMin/getMax

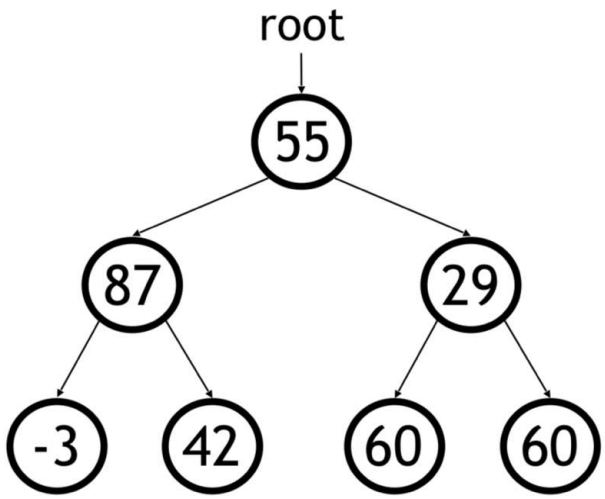
- Write functions **`getMin`** and **`getMax`** that accept a node pointer and return the minimum and maximum integer value from the tree. Assume that the tree is a valid non-empty binary search tree.

```
int min = getMin(root);   // 17
int max = getMax(root);   // 91
```



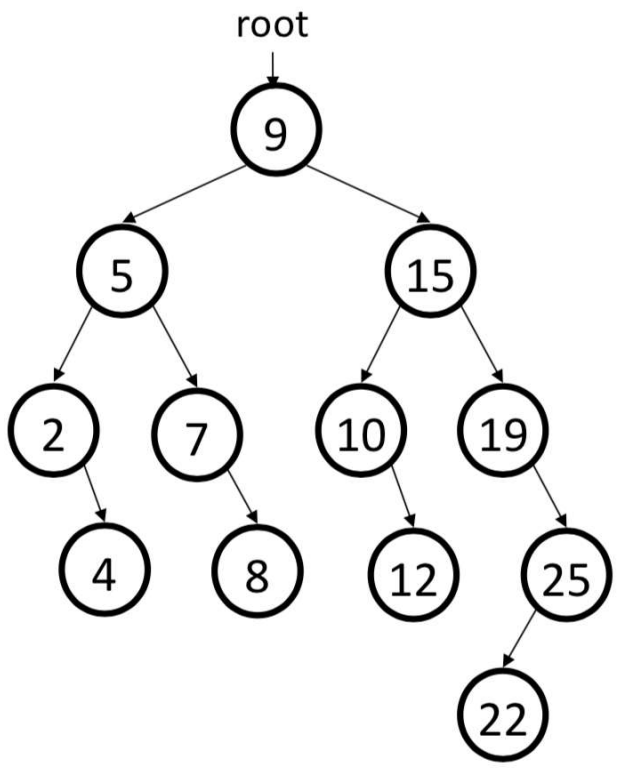
deleteTree

- Write a function **deleteTree** that accepts a tree node pointer as its parameter and frees all memory associated with the given tree.



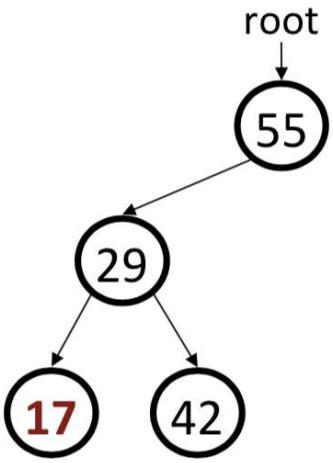
Removing from a BST

- Suppose we want to **remove** values from the BST below.
  - Removing a leaf like 4 or 22 is easy.
  - What about removing 2? 19?
  - How can you remove a node with two large subtrees under it, such as 15 or 9?
- What is the general algorithm?

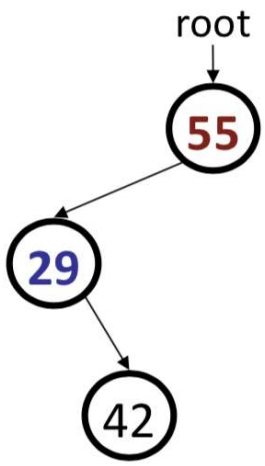


Removing from a BST

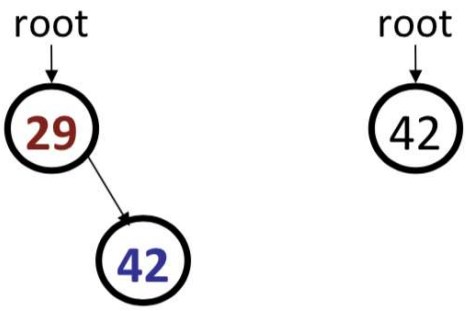
1. a **leaf**: Replace with `nullptr`
2. a node with a **left child only**: Replace with left child
3. a node with a **right child only**: Replace with right child



`remove(root, 17);`



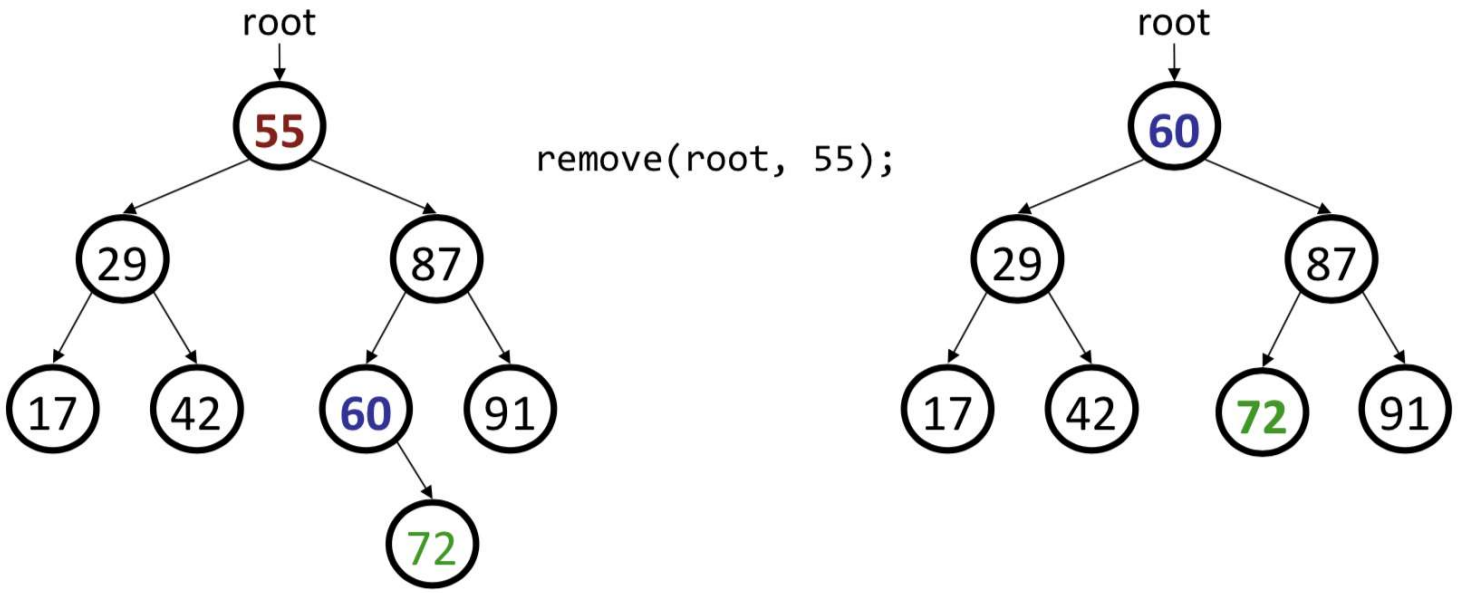
`remove(root, 55);`



`remove(root, 29);`

Removing from a BST

4. a node with **both** children: replace with **min from right**
(replacing with **max from left** would also work)



remove

- Add a function **`remove`** that accepts a root pointer and removes a given integer value from the tree, if present. Remove the value in such a way as to maintain BST ordering.

  - `remove(root, 73);`
  - `remove(root, 29);`
  - `remove(root, 87);`
  - `remove(root, 55);`

